

Data Analysis with Python 2024–2025

Parte 6: Machine Learning Types

Caderno de Exercícios

Tradução e Adaptação: University of Helsinki — MOOC.fi

Nota Importante

Este material é uma tradução e adaptação baseada no curso *Data Analysis with Python 2024-2025*, oferecido pela **Universidade de Helsinque (MOOC.fi)**.

Conteúdo

1 Exercícios - Capítulo 6 (Classificação Naive Bayes)

Exercício 6.1 – Classificação de Blobs (blob_classification)

Sumário

- **Objetivo:** Escrever a função `blob_classification(X, y)` que recebe a matriz de *features* `X` e o vetor de rótulos `y` e realiza uma classificação do tipo *Naive Bayes Gaussiano*.
- **Procedimento:**
 1. Use a função `train_test_split` da biblioteca `sklearn.model_selection` para separar os dados em conjunto de treino e conjunto de teste.
 2. Configure `random_state=0` para a separação dos dados de forma que o resultado seja sempre determinístico.
 3. Configure `train_size=0.75` (ou seja, 75% dos dados serão usados para treino e 25% para teste).
 4. Instancie o classificador `GaussianNB`, faça o fit do modelo com os dados de treinamento e utilize o `predict` com os dados de teste.
- **Retorno:** A função deve retornar o `accuracy_score` da previsão realizada.

Exercício 6.2 – Classificação de Plantas (plant_classification)

Sumário

- **Objetivo:** Escrever a função `plant_classification()` focada em prever o famoso *Dataset Iris* usando Naive Bayes.
- **Procedimento:**
 1. Carregue o conjunto de dados chamando `sklearn.datasets.load_iris()`.
 2. Divida os dados chamando `train_test_split` com tamanho de treino equivalente a 80% (`train_size=0.8`) e `random_state=0`.
 3. Ajuste (`fit`) o modelo aos dados utilizando o `GaussianNB`.
 4. Faça a previsão (`predict`) sob a parcela de testes.
- **Retorno:** O score exato da predição retornado pela função `accuracy_score`.

Exercício 6.3 – Classificação de Palavras (word_classification)

Sumário

- **Objetivo:** Construir um modelo para prever se palavras desconhecidas pertencem ao dicionário finlandês ou ao inglês. Este exercício exige que você processe a limpeza dos dados, conte letras e aplique Validação Cruzada (*Cross-Validation*).
- **Parte 1 - get_features(a):** Receba um array unidimensional de palavras e retorne a matriz de *features* no formato (n, 29). Para cada palavra, a respectiva linha da matriz deve conter o número de ocorrências de cada um dos 29 caracteres possíveis do alfabeto: "abcdefghijklmnopqrstuvwxyzäö-".
- **Parte 2 - contains_valid_chars(s):** Crie uma função auxiliar que retorne **True** apenas se a palavra **s** for composta unicamente pelos 29 caracteres listados acima.
- **Parte 3 - get_features_and_labels():** Utilize as rotinas que injetam a carga das palavras finlandesas e inglesas (você pode assumir que as listas são carregadas via métodos locais). Transforme o finlandês para letras minúsculas e retire palavras contendo caracteres inválidos (utilizando a Parte 2). Para palavras inglesas, **remova-as primeiro se começarem com letra maiúscula** (para excluir nomes próprios) e depois aplique as mesmas transformações usadas no finlandês (minúscula e caracteres válidos). Retorne (X, y) sendo finlandês a classe 0 e inglês a classe 1.
- **Parte 4 - word_classification():** Extraia X, y. Crie o classificador MultinomialNB. Em seguida, obtenha a matriz de avaliações a partir de `sklearn.model_selection.cross_val_score` aplicando um `KFold(n_splits=5, shuffle=True, random_state=0)`. Retorne a lista originada pelo `cross_val_score` com as 5 avaliações de acurácia geradas para esse conjunto.

Exercício 6.4 – Detecção de Spam (`spam_detection`)

Sumário

- **Objetivo:** Ler e identificar em arquivos brutos compactados se o texto base trata-se de um *Spam* ou de conteúdo limpo (*Ham*).
- **Procedimento:**
 1. Abra via módulo nativo do Python `gzip.open()` os arquivos `src/ham.txt.gz` e `src/spam.txt.gz`. Leia o conteúdo das linhas.
 2. Consuma uma quantidade fracionária dos arquivos baseada no parâmetro `fraction=[0.0, 1.0]`. Você pegará apenas as primeiras linhas correspondentes a esse percentual de arquivo para cada base lida (ex: `fraction=0.1` lê os primeiros 10% de cada arquivo).
 3. Junte as amostras de treino `ham` seguidas das amostras `spam` e passe-as em massa pelo `CountVectorizer().fit_transform` para converter em contagens vetorizadas de palavras.
 4. Divida os vetores transformados via `train_test_split` usando a amostra de 75% para treino e `random_state=random_state`. Atribua rótulos 0 (`ham`) e 1 (`spam`).
 5. Aplique o algoritmo `MultinomialNB`.
- **Retorno:** A função `spam_detection` deverá retornar uma tupla contendo o `accuracy_score` aferido na previsão, a quantidade de elementos testados, e o total absoluto (`int`) de mensagens testadas que receberam o rótulo incorreto no fim do teste.